COMP 699 – Professional Seminar Software Requirements and Architecture

1. Problem Definition and Project Scope

1.1 Problem Description

Students and academic researchers often need to locate specific information in lengthy academic research papers. Manually reading entire PDFs can be time-consuming, while keyword searches may miss relevant content when questions use different wording.

The proposed Academic Research Paper Assistant will allow users to upload an academic PDF and ask questions in natural language. A Retrieval-Augmented Generation (RAG) pipeline will retrieve relevant document passages and provide them with context to a Large Language Model (LLM) to generate answers. Source and page information will be displayed when available to help users verify the answers.

1.2 Intended Users and Stakeholders

The primary users are students and academic researchers who need to understand or locate information in research papers. The project developer designs, implements, and tests the application. The faculty assesses its requirements, architecture, implementation, and technical feasibility.

1.3 Project Objectives

The system will:

- Accept and process academic PDFs.
- Generate and store document embeddings.
- Retrieve relevant passages for natural-language questions.
- Generate answers grounded in a retrieved context.
- Display supporting source information when available.

COMP 699 – Professional Seminar Software Requirements and Architecture

1.4 Project Scope

In scope: PDF upload and text extraction, text chunking, embedding generation, vector storage, semantic retrieval, RAG-based question answering, source display, web interface, and testing.

Out of scope: Complete research-paper writing, plagiarism detection, training a new LLM, audio/video processing, enterprise administration, large-scale production deployment, and full Internet-based literature discovery.

2. Users and Stakeholders

2.1 Student or Researcher

- **Role:** Primary user.
- **Goals:** Find and understand information in academic PDFs.
- **Tasks:** Upload a paper, ask questions, review answers, and check supporting sources.
- **Constraint:** The interface should not require knowledge of AI implementation.

2.2 Project Developer

- **Role:** Designs, implements, tests, and documents for the application.
- **Goal:** Build a working RAG-based application within the Fall 2026 semester.
- **Tasks:** Develop and integrate the processing pipeline, interface, retrieval, and answer generation.
- **Constraint:** Use available resources and keep the project achievable within the semester.

2.3 Faculty Evaluator

- **Role:** Evaluates project requirements, architecture, implementation, testing, and results.
- **Goal:** Determine whether the project demonstrates meaningful software development and meets course requirements.

COMP 699 – Professional Seminar Software Requirements and Architecture

3. Functional Requirements

- **FR-01:** The system shall allow users to upload academic PDF documents through the web interface.
- **FR-02:** The system shall validate uploaded files and reject invalid or unsupported PDFs.
- **FR-03:** The system shall extract text from uploaded PDFs.
- **FR-04:** The system shall divide extracted text into smaller chunks for retrieval.
- **FR-05:** The system shall generate embeddings for document chunks.
- **FR-06:** The system shall store embeddings and associated metadata in a vector store.
- **FR-07:** The system shall accept natural-language questions about an uploaded document.
- **FR-08:** The system shall retrieve document chunks relevant to a question.
- **FR-09:** The system shall provide retrieved content to the LLM as context.
- **FR-10:** The system shall generate answers based on the retrieved context.
- **FR-11:** The system shall display the user's answers**.**
- **FR-12:** The system shall display relevant source passages and page information when available.
- **FR-13:** The system shall display appropriate error messages when processing or question answering fails.
- **FR-14:** The system shall provide a web interface for document uploads and questions.

4. Nonfunctional Requirements

- **NFR-01 — Performance:** Record document-processing and question-answering times during testing.
- **NFR-02 — Usability:** Provide a straightforward interface for uploading PDFs, asking questions, and reviewing answers without requiring technical AI knowledge.
- **NFR-03 — Reliability and Error Handling:** Handle corrupted or invalid PDFs with clear error messages rather than crashing the application.
- **NFR-04 — Compatibility and Portability:** Document installation, configuration, dependencies, and execution instructions for the intended environment.

- **NFR-05 — Maintainability:** Separate document processing, embedding, storage, retrieval, and answer generation into logical components.
- **NFR-06 — Cost and Resources:** Prioritize free, locally runnable tools where practical and evaluate their feasibility during the PoC.
- **NFR-07 — Answer Traceability:** Display supporting passages and document or page information when available so users can verify answers.

5. System Architecture

The application will use Python and Streamlit for the user interface, LangChain to organize the RAG workflow, ChromaDB for local vector storage, and Ollama with Llama 3.2 for answer generation.

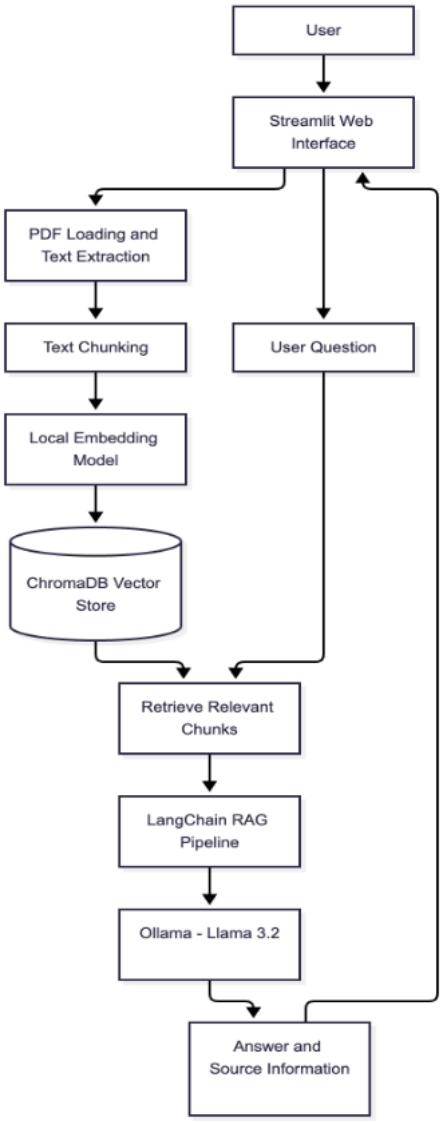
The main components are:

- **User Interface:** Streamlit handles PDF uploads, questions, answers, and source display.
- **Document Processing:** A PDF loader extracts text and available page metadata; a text splitter creates manageable chunks.
- **Embedding and Retrieval:** A compatible local embedding model converts chunks and questions into vectors. ChromaDB stores embeddings and metadata and retrieves relevant passages.
- **Answer Generation:** LangChain provides retrieved context to Llama 3.2 through Ollama to generate an answer.
- **Source Display:** The interface presents supporting passages and document or page information when available.

The initial design uses two-step RAG: retrieve relevant passages first, then generate an answer from those passages. A retrieval agent is an optional future enhancement. The selected components will be validated during the PoC.

Figure 1. System Architecture of the Academic Research Paper Assistant.

COMP 699 – Professional Seminar Software Requirements and Architecture



6. Data Design

The application will process academic PDFs and maintain text chunks, embeddings, and source metadata for retrieval. ChromaDB will store the embeddings and associated metadata locally; a relational database is not required for the initial implementation.

Core data entities:

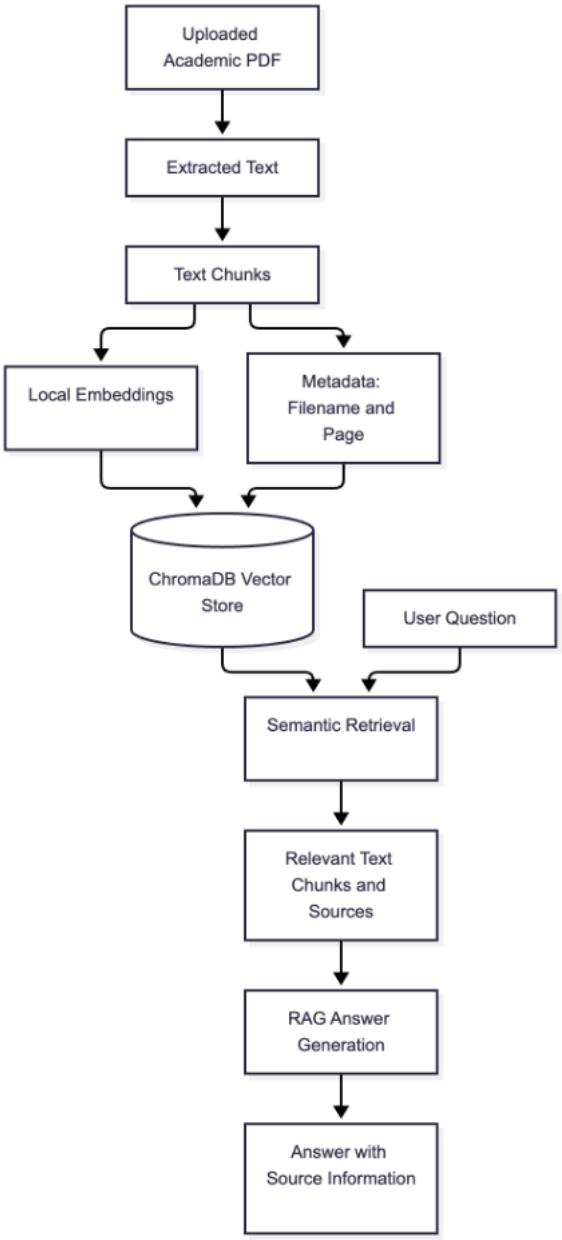
- **Raw Input:** User-uploaded academic PDF.

- **Text Chunks:** Segments extracted from the document.
- **Metadata:** Source filename and page number, when available.
- **Vector Store:** ChromaDB embeddings and associated metadata.

The storage configuration and preservation of source information will be checked during the PoC.

Figure 2. Document Processing and Data Flow.

COMP 699 – Professional Seminar Software Requirements and Architecture



7. Interfaces and APIs

The application will use a Streamlit browser interface with three main elements:

- **Document Upload:** Accepts academic PDFs and displays errors for invalid or unreadable files.

- **Question Input:** Allows users to ask natural-language questions about the uploaded paper.
- **Answer and Source Display:** Presents the generated answer and supporting passages or page information when available.

LangChain will coordinate the internal Python components. Ollama will provide local model inference, and ChromaDB will handle vector storage and retrieval. The initial design does not require a separate public REST API.

8. Technology Stack and Design Decisions

Component	Planned technology and purpose
Programming language	Python — application and RAG logic
User interface	Streamlit — lightweight browser interface
RAG framework	LangChain — document processing and retrieval workflow
LLM runtime	Ollama with Llama 3.2 — local answer generation
Embedding model	Local model to be selected based on compatibility and PoC results
Vector store	ChromaDB — local embedding and metadata storage
Development tools	VS Code, Git, and GitHub — development and version control

The design prioritizes local processing, low cost, and manageable implementation complexity. Streamlit avoids a separate frontend framework, while ChromaDB avoids a hosted vector database. Model compatibility and performance will be evaluated during the PoC.

10. Technical Proof of Concept (PoC)

The PoC will assess whether the proposed local RAG workflow can operate with the selected tools.

Objectives: Verify PDF text extraction, chunking, local embedding generation, ChromaDB storage and retrieval, and answer generation through Ollama without relying on external cloud services.

COMP 699 – Professional Seminar Software Requirements and Architecture

Approach: Process a sample academic PDF, store its embedded chunks and available source metadata, and submit natural-language questions to test retrieval and answer generation.

Initial Setup Results: The required Python packages have been installed and verified, and the local Ollama runtime with Llama 3.2 has been successfully tested through LangChain. These results confirm that the development environment is ready for implementation. Figures 3 and 4 provide evidence of these setup checks. The complete PDF-to-answer RAG workflow will be implemented and evaluated during the planned PoC phase.

Figure 3. Verification of installed Python dependencies.

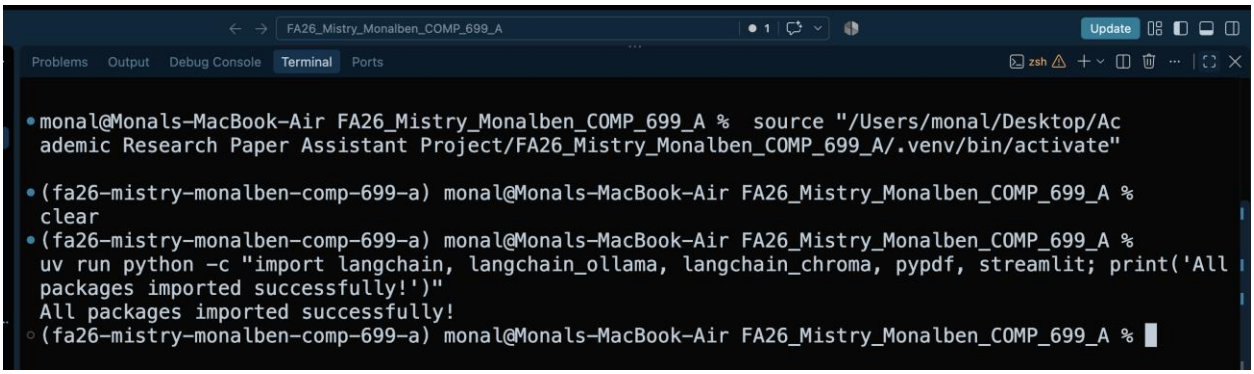
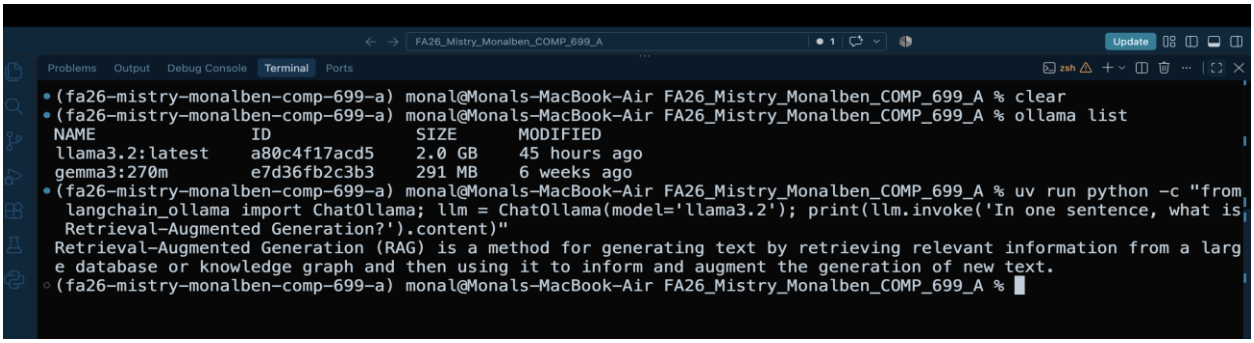


Figure 4. Verification of Ollama Llama 3.2 and its integration with LangChain.



Results and Conclusions: After implementation, record actual outputs, processing and response times, source traceability, and limitations. Use these findings to guide integration into the Streamlit application.

COMP 699 – Professional Seminar Software Requirements and Architecture

11. Testing and Evaluation Plan

Testing will evaluate the functional and nonfunctional requirements through the following activities:

- **Functional and Integration Testing:** Verify PDF processing, retrieval, and answer generation.
- **Retrieval and Answer Validation:** Use factual questions about a sample academic paper to assess passage of relevance and whether answers are supported by the document.
- **Reliability Testing: Test invalid or unreadable PDFs and verify clear errors are shown.**
- **Performance Evaluation:** Record document-processing and question-answering times under normal local conditions.

Record test cases expected and actual results, and pass/fail status. Address identified issues before the final evaluation.

12. Risks and Technical Challenges

- **Local Model Performance:** Local inference may be slow or resource intensive. Evaluate performance during the PoC.
- **Retrieval and Answer Accuracy:** Retrieved passages may be incomplete, and answers may contain unsupported claims. Test representative questions and display source passages.
- **PDF Text Extraction:** Scanned or unreadable documents may fail extraction. Provide clear error messages; OCR is outside the initial scope.
- **Technology Compatibility:** LangChain, Ollama, ChromaDB, and the embedding model may require configuration adjustments. Test compatibility early.
- **Time and Scope Constraints:** Prioritize the core two-step RAG workflow and keep advanced features, such as retrieval agents, optional.

COMP 699 – Professional Seminar Software Requirements and Architecture

13. Development Schedule

The project will be developed during the Fall 2026 semester (September 8–December 18, 2026).

- **Phase 1: Architecture and Design (September 23–October 16):** Complete requirements, confirm the technology stack, and finalize both diagrams.
- **Phase 2: Technical Proof of Concept (October 17–30):** Test the local PDF-to-answer workflow, record actual results, and maintain the PoC code in GitHub.
- **Phase 3: Core Implementation (October 31–November 7):** Develop the core Python components for document processing, embeddings, retrieval, and answer generation.
- **Phase 4: Component Integration (November 8–14):** Integrate Streamlit, LangChain, ChromaDB, and Ollama.
- **Phase 5: Minimum Viable Product (MVP) (November 15–20):** Prepare a functional prototype for uploading a PDF, asking questions, receiving answers, and viewing available sources.
- **Phase 6: Testing and Evaluation (November 21–27):** Evaluate functionality, grounding, traceability, error handling, and response times.
- **Phase 7: Refinement (November 28–December 4): Fix issues and improve usability and reliability.**
- **Phase 8: Documentation (December 5–11):** Finalize the README, setup instructions, dependency and configuration information, and test results.
- **Phase 9: Final System and Presentation (December 12–18):** Verify the final prototype and prepare the demonstration and submission materials.

The schedule may be adjusted based on PoC findings and technical challenges. Core RAG functionality will take priority over optional enhancements.